

Ring Election Protocol

Learning goal. Follow the election token around an ID-ordered ring, understand candidate selection and ACK timeouts, and see how the code suppresses competing elections.

Implementation inspected: `feature/electionTransaction` at `d1222a2`.

Key files: `ElectionTransaction.java`, election sections of `Replica.java`, `TestElectionTransaction.java`, election diagrams.

1. What election must decide

After coordinator failure, replicas must choose one new coordinator. This is not simply “highest replica ID wins.” The winner should be the surviving replica with the freshest observed update so it has the best recovery knowledge. Replica ID breaks equal-freshness ties.

The logical ring sorts numeric replica IDs. Physical actor creation order or map iteration order does not define the ring.

2. Candidate ordering

`ElectionCandidate` contains:

- numeric replica ID;
- whether it has observed any update; and
- latest observed `EpochPair` when present.

Comparison rules, from strongest to weakest, are:

- 1 a candidate with real observed-update information beats one with none;
- 2 newer epoch pair beats older;
- 3 higher replica ID breaks a tie.

Examples:

```
R2 at <3,9> beats R4 at <3,8>
R4 at <3,9> beats R2 at <3,9>
R1 with <0,0> beats R5 with no observed update
```

The candidate list is copied into `ElectionMsg`, preventing mutation while the token is in flight.

3. Message vocabulary

- `ElectionStartMsg`: local delayed trigger after failure detection.
- `ElectionMsg`: network token carrying failed coordinator ID and candidates.
- `ElectionAckMsg`: confirms the next replica received/forwarded the token.
- `ElectionAckTimeoutMsg`: local attempt timeout with expected target and version.
- `ElectionRejectMsg`: tells a losing competing initiator to cancel.
- `SynchronizationMsg`: leadership/recovery announcement, covered mainly in the recovery report.

4. Local FSM states

```
NEW -> PARTICIPATING -> ELECTED -> SYNCHRONIZING -> DONE
      \-----> DONE on rejection/failure
```

PARTICIPATING covers both an initiator and a replica that joined an incoming token. ELECTED means this local candidate won. SYNCHRONIZING acknowledges that leadership is not operational until recovery work is initiated.

5. Staggered election start

Every follower can notice the same failed coordinator. Starting all tokens simultaneously would create unnecessary competition.

`Replica.startElection` calculates ring distance from the failed coordinator and schedules start after:

```
ringDistance * maxLatencyPlusTolerance
```

The closest successor starts first. Sets prevent repeated scheduling, repeated participation, or restarting an already completed failure term.

Staggering reduces competition; it does not eliminate it. The code still defines deterministic competition resolution.

6. Healthy-ring trace

Suppose R1 failed and ring survivors are R2, R3, R4.

- 1 R2 begins first, creates a negative-sequence election transaction ID, and places its local candidate in the token.
- 2 R2 sends token to R3 and schedules ACK timeout for that target/attempt.
- 3 R3 adds its candidate, forwards to R4, then ACKs R2.
- 4 R4 adds its candidate, forwards to R2, then ACKs R3.
- 5 R2 sees its own candidate already present: the collection phase completed a ring.
- 6 It computes the best candidate.
- 7 If R2 is winner, it ACKs previous sender and starts winner completion. Otherwise, it sends the full candidate list directly toward the winner.
- 8 The winner recognizes itself and starts synchronization.

Forward-before-ACK ensures that receiving an ACK means the next participant accepted responsibility for continuing the token.

7. Skipping a crashed neighbor

When a sender forwards, it stores:

- pending target ID;
- pending message;
- incremented attempt version; and
- timeout message containing target/version.

If the expected ACK does not arrive:

- 1 reject stale timeout if target/version no longer match;
- 2 mark the target unavailable;
- 3 remove that target's candidate evidence from the pending list;

- 4 choose the next available ring ID; and
- 5 resend with a new attempt version.

Removing a crashed candidate matters. Otherwise a dead but freshest candidate could be elected forever.

8. ACK validation

An ACK is accepted only if there is a pending message and its sender maps to the current pending target. Valid ACK cancels the timer and clears pending state.

The attempt version protects against this race:

```
attempt 4 to R3 times out
attempt 5 starts toward R4
old timeout or late event from attempt 4 arrives
```

Only attempt 5 may affect current forwarding.

9. Competing elections

`Replica` tracks the current election transaction ID per failed coordinator. If another token arrives, it compares initiators:

- lower numeric initiator ID is preferred;
- for the same initiator, lower election sequence number is preferred.

Election IDs use negative sequence numbers, separated from ordinary non-negative replica transactions and heartbeat sequence zero.

The preferred incoming election replaces the current transaction. A non-preferred token is rejected. An initiator receiving `ElectionRejectMsg` completes and unregisters its losing FSM.

10. Input validation

Before creating/routing a token, `Replica.isValidElectionMessage` rejects:

- null transaction/sender/initiator;
- senders or initiators outside membership;
- unknown failed coordinator;
- the failed coordinator trying to participate locally;
- candidates outside membership;
- the failed coordinator in candidates; and
- duplicate candidate IDs.

Validating before routing prevents malformed messages from poisoning transaction maps or causing null dereferences.

11. Termination reasoning

The ring contains a finite static set. Each failed forwarding target is added to `unavailableReplicaIds`, so a local attempt does not retry the same dead target forever. A strict majority survives, so at least one candidate should remain.

The branch also removes a selected winner if it cannot be contacted and recomputes from remaining candidates. This addresses the case where the best candidate crashes after token collection.

End-to-end termination still depends on timeout accuracy, valid membership, and integration with heartbeat/crash behavior.

12. Current scope limitations

- Candidate freshness uses the local `epochPair`; integrated immutable update history is absent.
- Synchronization is a snapshot broadcast rather than full update replay.
- There is no explicit global election deadline beyond per-hop progress and competition rules.
- The branch's focused tests emphasize data structures/validation and do not simulate every healthy/crashing ring sequence end to end.

13. Test map

`TestElectionTransaction` checks:

- epoch/sequence/ID candidate ordering;
- no-update candidate behavior;
- ring wraparound and skipped IDs;
- message immutability;
- ACK and timeout metadata;
- synchronization defensive copy/validation; and
- malformed election rejection.

Needed additional tests include full token circulation, neighbor crash skip, best-candidate crash, two initiators converging, and exactly-once callbacks on all survivors.

14. Exam rehearsal

Why use freshness before replica ID? The new coordinator must possess the best recovery knowledge. ID is only a deterministic tie-breaker.

Why ACK after forwarding? It transfers responsibility for progress; the previous sender may stop retrying only after the receiver has continued the token.

The invariant to remember is: **for each failed coordinator, correct replicas converge on one surviving election token and one best available candidate, while each failed next hop is eventually skipped.**

15. Walk a token around a five-replica ring

```
R0 -> R1 -> R2 -> R3 -> R4 -> R0
|                               |
+-- election token: candidate=R0, observed=(2,4)

R1 compares its state, keeps newer candidate if any, forwards
R2 does the same ... R4 forwards to R0
R0 sees its own token again -> install winner -> synchronize
```

The ring gives each survivor a chance to contribute recovery knowledge. It is not a broadcast tree: a crashed next hop must be skipped, and an ACK means the next participant accepted responsibility for forwarding. The election result is still incomplete until the synchronization phase installs a consistent snapshot/history.

16. Code microscope: candidate comparison and token identity

Candidate comparison should be deterministic and explainable: prefer a non-empty observed epoch, then a newer EpochPair, then a higher replica ID as a tie-breaker. A token also needs an attempt/generation so an old ACK cannot advance a newer circulation after a timeout.

```
if (incoming.observed().isNewerThan(best.observed()))
    best = incoming;
else if (sameEpoch(incoming, best)
        && incoming.replicaId() > best.replicaId())
    best = incoming; // deterministic tie-break

if (ack.attempt() != currentAttempt) return; // stale ACK
forwardToNextLive(token.with(best));
```

When a candidate or next hop is dead, the sender needs a bounded traversal strategy. Blindly retrying a dead actor can keep the mailbox busy forever; skipping it without recording the skip can make the proof of ring coverage unclear. Tests should observe both the skip and eventual completion.

17. Competing elections and convergence

Two followers may detect silence close together and start tokens. The protocol must make both circulations converge on one winner rather than install two coordinators. Typical tools are a deterministic candidate order, token identity, and forwarding rules that absorb an inferior token. Draw two tokens crossing the same edge and mark which one survives; if the design cannot explain that picture, it is not yet a convergence argument.

Practice

Create a table for replicas with observed epochs (3, 8), (3, 10), and (4, 0) and IDs 1–3. Which candidate wins, and why? Then repeat with the best candidate crashed before synchronization and explain what the recovery layer must do.